

人芯对话：昇腾950 算力落地算子的编程范式探索

<https://gitcode.com/cann>

CANN

目录

Part 1 Ascend C

Part 2 PyPTO

Part 3 CANNBot

释放每一滴算力 --- Ascend 950 x Ascend C 新同构编程

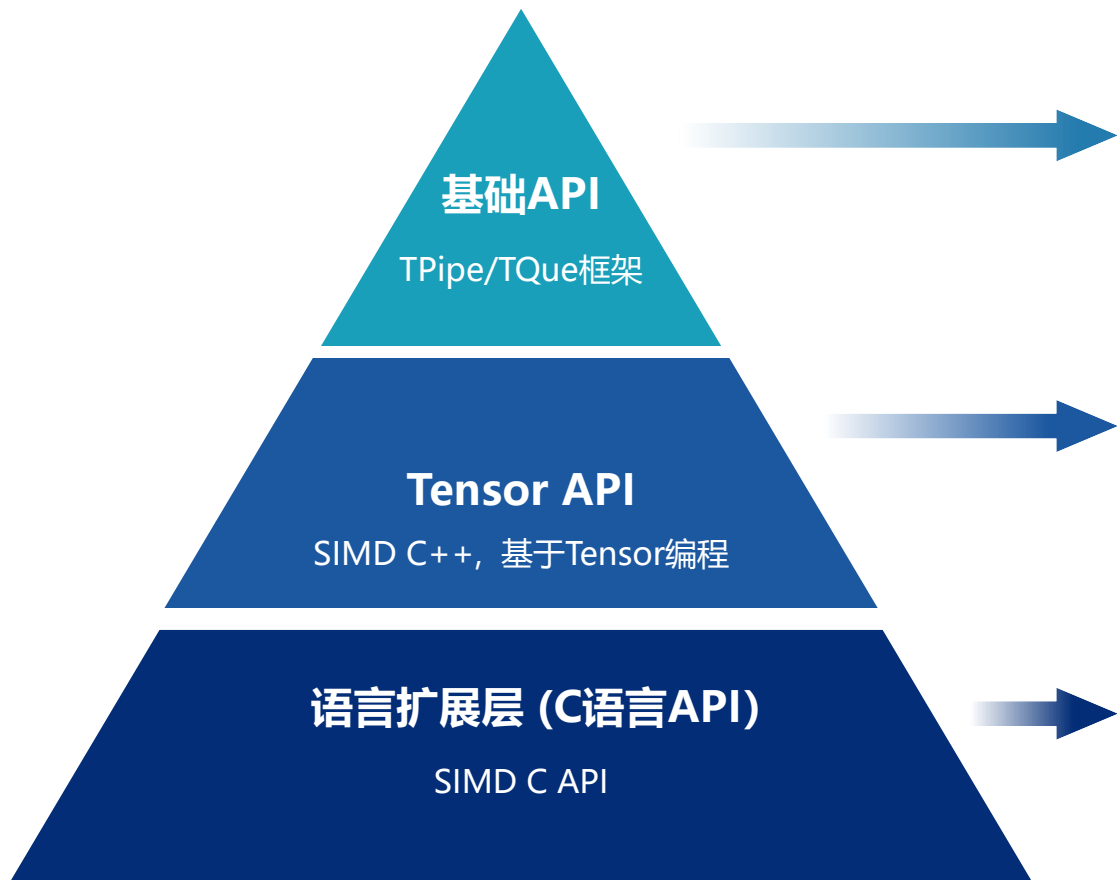
- **SPMD + SIMD**: 将一个AI Core抽象成一个逻辑核 (Block) , 所有Block运行相同代码, 以Block为视角编程, Block内部调用SIMD指令;
- **SPMD + SIMT**: 将硬件抽象成Grid->Thread Block->Thread架构, 以Thread为视角进行编程, 关注每个Thread需要处理的数据;
- **SPMD + SIMD/SIMT**: 核函数本质遵循SPMD+SIMD, 核函数内部以函数 (Vector Function) 为粒度进行SIMD与SIMT切换; 一个SIMT Vector Function抽象成一个SIMT Thread Block, 内部基于Thread编程; 一个SIMD VF内部调用SIMD指令。

编程模型	支持范围	适用场景	芯片支持
SIMD(主)	向量、矩阵、融合计算	规整、高密度任务 (卷积、矩阵乘、逐元素变换)	昇腾全系列
SIMT(辅)	仅向量计算	离散访问、复杂分支、稀疏计算	Ascend 950PR/Ascend 950DT
SIMD+SIMT混合	向量、矩阵、融合计算	用SIMT处理不规则逻辑, 用SIMD批量高效计算	Ascend 950PR/Ascend 950DT

- 开发常规高性能算子 (如矩阵乘、逐元素算子) 时, 建议使用SIMD编程模型。该模型支持昇腾全系列产品, 且具有较高的能效比。
- 对于涉及离散数据或复杂分支逻辑的算子, 建议使用SIMT编程模型。

同一套语言，多个选择 --- 新增 C API 和 Tensor API 双引擎设计

秉持「标准C/C++语法、最小化扩展」的核心设计原则，打造轻量化、高性能的底层编程基座



03 | TPipe/TQue框架编程API

基于Tensor编程，通过TPipe/TQue自动管理内存搬运与同步，屏蔽底层细节；使用框架自动编排数据搬运和计算，提升编程易用性与开发效率

02 | 基于Tensor编程，提供C++完备编程能力

- **核心框架：**Tensor/Layout代数体系 + Arch/Atom/Algorithm API层级；
- **核心特点：**张量与布局为一等公民、零成本编译期抽象、三层解耦，关注点分离、天然架构可移植性

01 | 基于指针编程，提供C完备编程能力

- 统一命名风格，匹配C语言开发习惯；
- 原生指针编程；
- 指令级透明映射，零封装开销；
- 开放全部底层硬件能力，支撑精细化调优与极致性能实现

摸到硬件的脉络 --- C API指针编程的特性与优势

匹配C 语言开发习惯，提供了硬件底层的编程自由与控制能力

01 零开销原则 (Zero-overhead)

接口与硬件指令一一对应，无额外运行时封装与隐式调度，性能损耗完全透明、可量化预测，拒绝“黑盒”损耗。

02 原生指针编程 (Native Pointers)

支持开发者自主管理多级内存的分配、搬运与同步，打破高层抽象限制，实现对数据流向与算力调度的极致掌控。

03 命名风格统一 (Unified Style)

全系采用标准 snake_case 命名规范，深度贴合系统级 C 语言编程习惯，降低底层开发的认知负荷与代码迁移成本。

04 指令级透明映射 (Transparent Mapping)

单个 C API 直接映射单条硬件原语，无中间层转译损耗，确保每一行代码的执行效果精准可控，调试更高效。

 **极致算力场景：**大模型推理、科学计算等需榨干硬件算力的头部业务，突破性能天花板。

 **硬核开发群体：**面向精通底层架构，追求完全自主控制同步逻辑与内存布局的资深开发者。

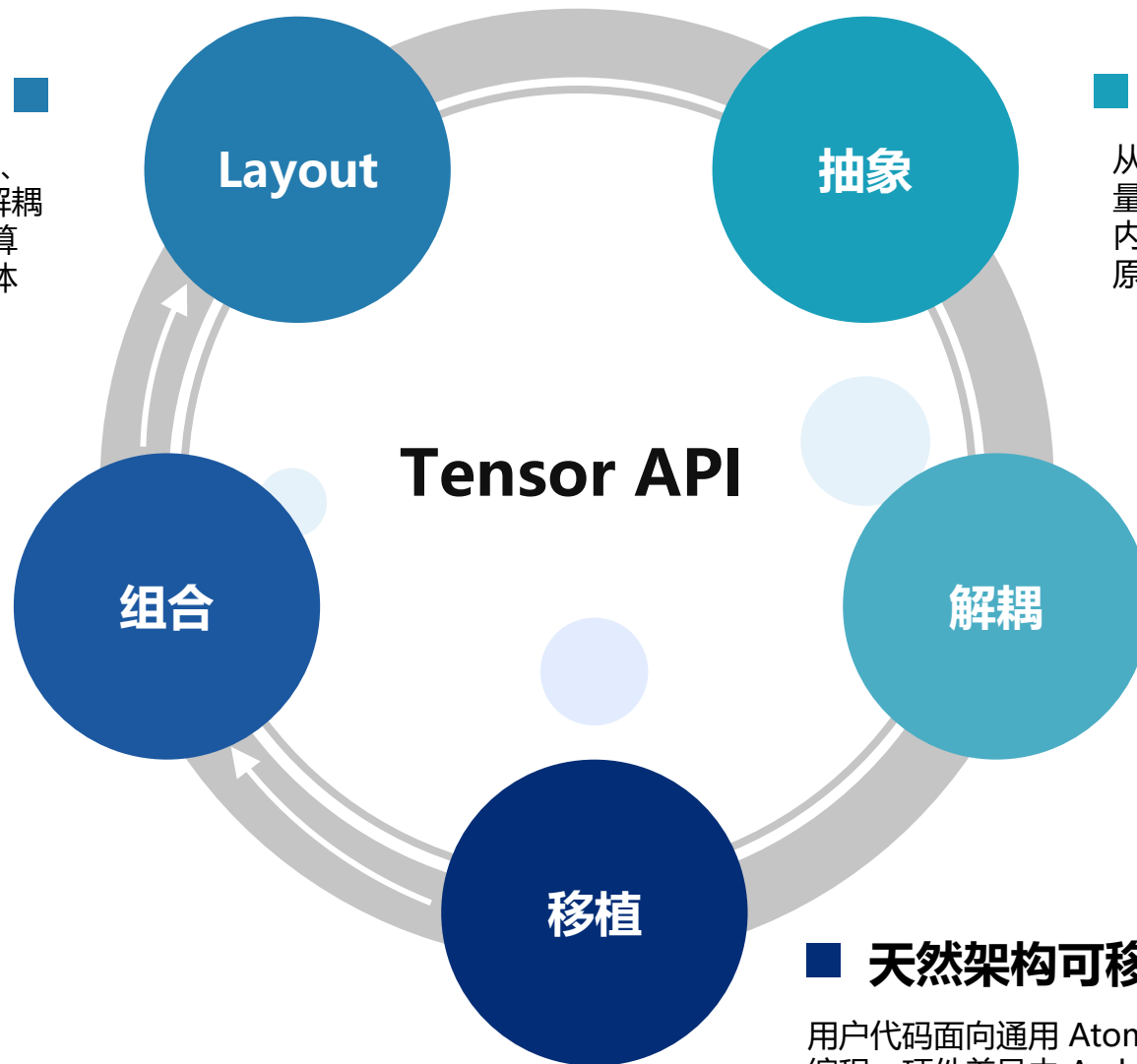
专注算法的灵魂 --- Tensor API的Layout抽象与开发效率

张量与布局为一等公民

基于布局代数体系，将多维 Tensor、内存 Layout 作为核心编程对象；解耦逻辑视图与物理存储，无需手工计算多维数组地址偏移，大幅提升编程体验

乐高式高度可组合

任意 Atom 可适配任意 Algorithm，任意 Layout 可结合任意 Atom；通过模块化积木式拼接构建内核，大幅降低高性能 Kernel 开发成本



零成本编译期抽象

从 Arch 到 Algorithm 全层均为轻量编译期封装；无虚函数、无动态内存分配，运行时指令性能与手写原生 C 代码完全等价

三层解耦，关注点分离

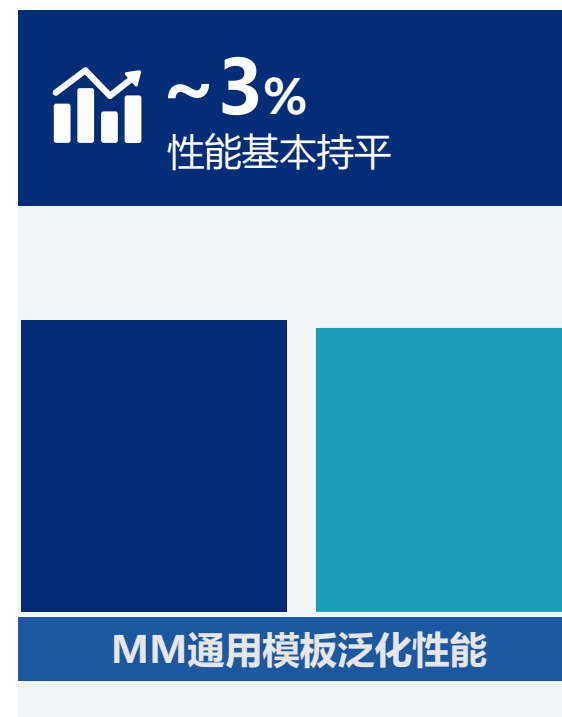
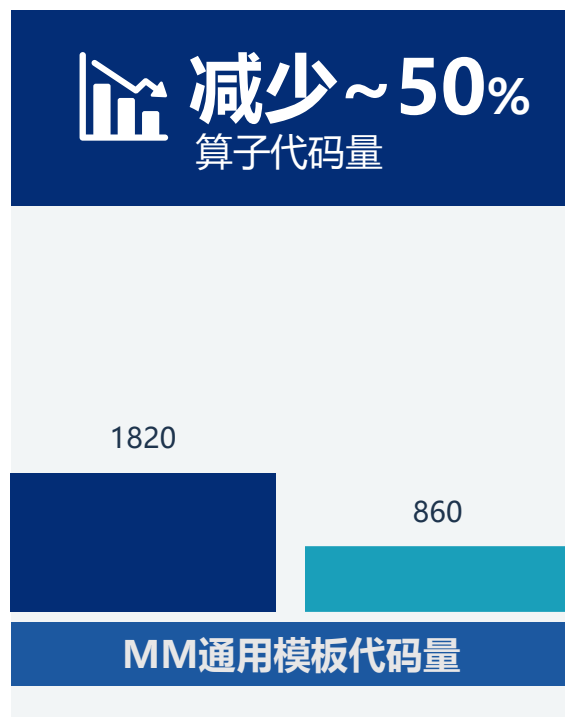
硬件指令 (Arch)、原子层 (Atom)、算法层 (Algorithm) 独立演化；新架构仅需新增 Arch 层特化，Atom 与 Algorithm 上层代码几乎无需修改

天然架构可移植性

用户代码面向通用 Atom / Algorithm 接口编程，硬件差异由 Arch 层特化屏蔽；较好支持跨架构、跨芯片的代码迁移与复用。

CANN

性能是测试出来的 --- Ascend 950基于Tensor API开发实践



基线版本 (手工计算)



基于Tensor API实现

目录

Part 1 Ascend C

Part 2 PyPTO

Part 3 CANNBot

PyPTO: 昇腾原生亲和 Python Tensor/Tile 编程

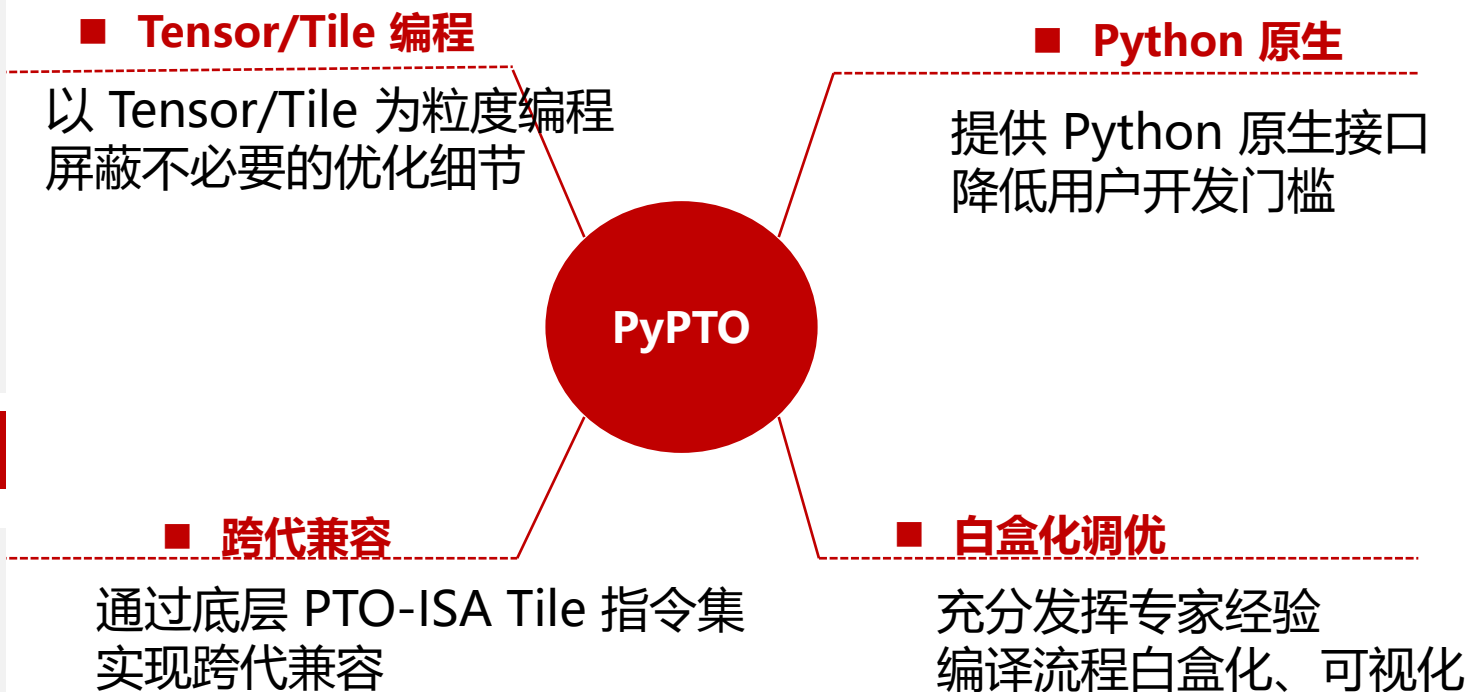
PyPTO 是什么?

- **本质:** 面向 AI 加速器的高性能编程框架
- **目的:** 简化算子开发流程, 同时保持高性能计算能力
- **范式:** 创新的 PTO 编程
- **理念:** 基于 Tensor/Tile 的编程模型

PyPTO 的定位

- 不仅限于单个算子的开发, 更强调**复杂融合算子, 甚至于整个模型网络**的开发能力
- 以**Tile块为最基础的数据结构抽象**, 以此来简化算子编程

<https://gitcode.com/cann/pypto>



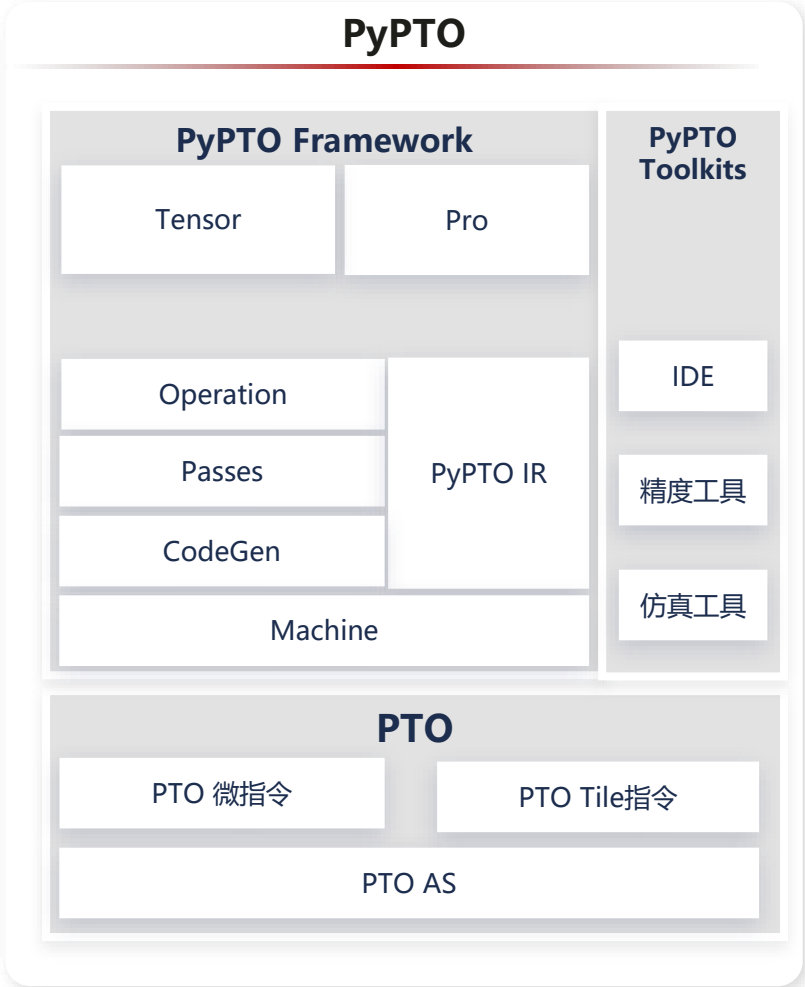
CANN

PyPTO：昇腾原生 Python Tensor/Tile编程架构

算子编程语言需要描述算子的**计算逻辑**、**调度策略**以及**指令映射**，而调度策略和指令映射是需要针对硬件做特殊设计的，实现难度较大。

- PyPTO-Tensor：创新性提出自动Tile切分+自动MPMD调度方案，前端只需要描述计算逻辑，**调度和指令映射交给框架**。易用性最好，但是由于调度开销和切分算法的局限性，性能做不到极致。
- PyPTO-Pro：暴露的 Tile 级别的buffer控制，通过一系列措施，简化了调度策略和指令映射的写作。补充了Tensor编程的控制力短板**

维度	PyPTO-Tensor	PyPTO-Pro
IR表达	PyPTO-IR	PyPTO-IR
前端抽象	Tensor-Level	Tile-Level
内存层级	不感知	感知
内存分配	不感知	手动分配
内存复用	不感知	手动复用
内存抽象	不感知	Tile + BufferManager
核内同步	不感知	不感知
CV自动分离	不感知	不感知
核间同步	不感知	不感知
Tile表达	多维	2维
Offset计算	传入坐标自动计算	传入坐标自动计算
Tensor表达	Shape + Stride+ Layout	Shape + Stride + Layout
VF融合	利用bisheng能力	利用bisheng能力
VF手写能力	支持	支持
Tiling表达	单tiling key	多tilingkey 二进制
性能	0.6+ 理论性能	0.9+ 理论性能



PyPTO-Tensor: 让开发复杂融合AI算子更简单

以Tensor描述“做什么”，以Tile保留“如何高效执行”的优化空间。

- ✓ **易开发**: Python友好API, 贴近数学表达。让用户关注做什么, 交由框架具体去设计怎么做。
- ✓ **易迁移**: 算法与硬件解耦, 快速适配不同硬件平台
- ✓ **符号化编程**: 内置调优开关, 根据不同场景灵活配置
- ✓ **自动Tiling**: 通过 `set_vec_tile_shapes` / `set_cube_tile_shapes` 设置基本块, 框架自动切分Tile感知并行与内存层次
- ✓ **MPMD 高效调度**: 不同核可在同一时间执行不同程序, 多核并行, 自动负载均衡, 不需要手排流水。支持编译、调度、DFX工具链协同

Python表达

```
@pypto.frontend.jit
```

```
def fused_op(q, k, v, result):
```

```
    qt = pypto.view(q, [x_len, y_len], [x_idx, y_idx])
```

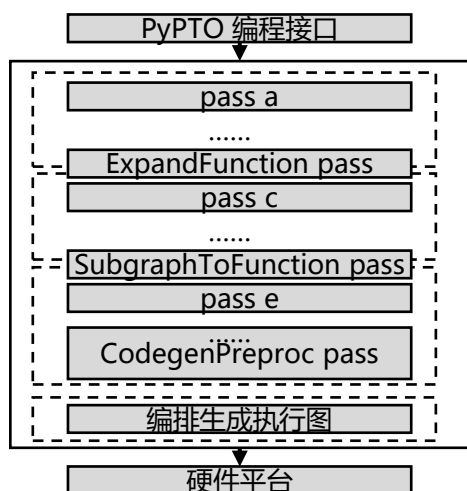
```
    kt = pypto.view(k, [x_len, y_len], [x_idx, y_idx])
```

```
    vt = pypto.view(v, [x_len, y_len], [x_idx, y_idx])
```

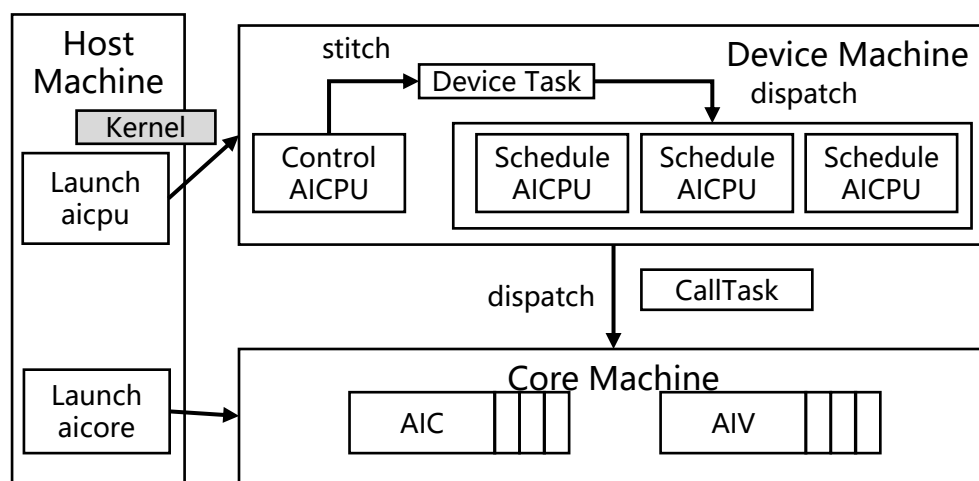
```
    score = pypto.matmul(qt, kt)
```

```
    value = pypto.matmul(score, vt)
```

```
    pypto.assemble(value, [x_idx, y_idx], result)
```



计算图编译流程



执行态运行示意图



PyPTO-Pro: Ascend 950 芯片算子编程的挑战

背景挑战

- PyPTO-Tensor 具有良好的易用性，但是缺少对硬件的细粒度掌控力，难以满足极致性能的要求。
- 传统极致性能算子编程中存在5大难点，需要选择性的解决

算子编程不可能三角

Generality



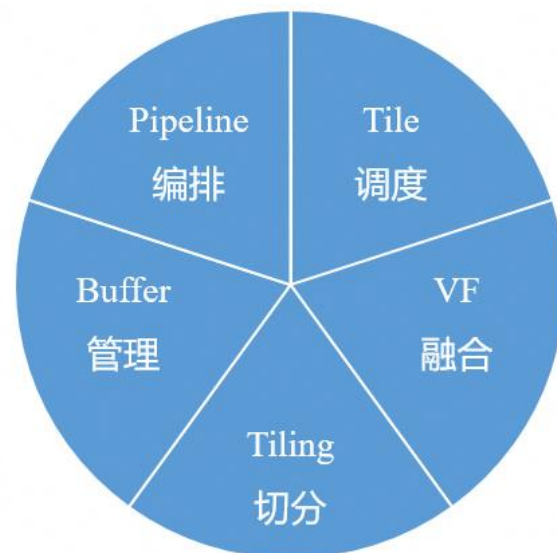
Usability

Efficiency

- 泛化性、易用性、高性能是算子编程领域的不可能三角，目前还不存在一款能够三者兼顾的编程框架
- PyPTO-Tensor编程致力于极致易用和极致的泛化，但是性能存在短板
- 通过汲取融合算子开发经验，PyPTO-Pro 寻求在泛化和高性能的前提下，追求 Kernel 代码编写的局部易用性

基于开发经验，聚焦核心易用性痛点

AI算子极致性能维度



主要难点

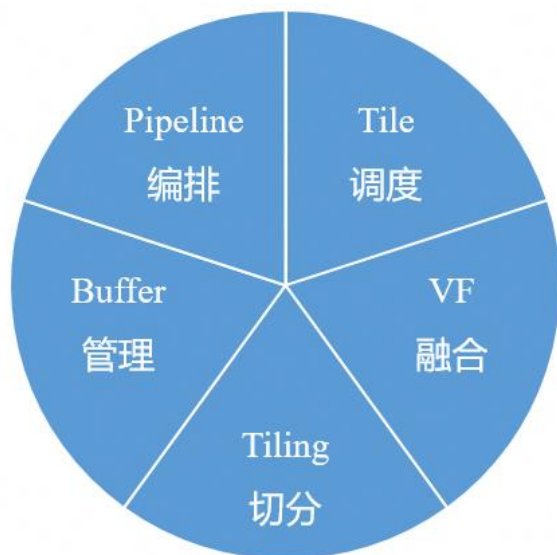
- Tiling切分：**
需要同时考虑单核内空间、计算效率、多核负载均衡等多个维度，设计难度大且算子差异大。
- Buffer管理(Buffer指代核内存储空间，如L1、UB)：**
Buffer复用算子差异大，高性能的复用需要结合Tiling一起设计，并结合上板性能数据调整；
多Buffer轮询代码和同步耦合，写作困难。
- VF融合：**
基于Regbase的Vector Function性能好，但是易用性不好。
- Pipeline编排：**
融合算子的核间流水编排需要最大化CV并行度；
流水场景下的核间同步插入难度大，易卡死。
- Tile块调度(Tile用于描述核内存储空间)：**
最小粒度的基本块的硬件操作需要感知硬件的硬核指令参数。

PyPTO-Pro: 致力于通过合理的取舍, 保留易用性的同时获取高性能

核心 愿景

- 极致性能: 开发者仍然拥有对硬件几乎全部的控制力, 可以极限压榨硬件性能;
- 局部易用: 基于多年算子开发经验, 将不影响性能指令和写法充分封装, 最大化提升易用性;
- Agent友好: 经过Tile封装后的前端接口更有利于Agent去理解, 可以使用Agent加速开发流程。

AI算子极致性能维度



五大维度重新审视

- Tiling切分: 对性能影响大, 自动化算法做不到极致性能, 仍然交给用户去设计, 不做封装。
- Buffer管理: 对于Buffer的复用, 性能影响大, 且需要性能数据支持并和Tiling切分紧密绑定, 不做封装; 对于Buffer的轮询使用, 同步插入可以封装
- VF融合: 提供Tile级别的Vector操作接口实现自动VF融合, 简化Vector算子实现; 这一步有性能损失, 对于Vector Bound算子, 也提供了Register-Base的写法。
- Pipeline编排: 框架提供自动核间流水编排的能力, 用户只需要写串行代码
- Tile块调度: 框架将硬件指令封装成Tile粒度的操作 代码量更少且更易于理解。

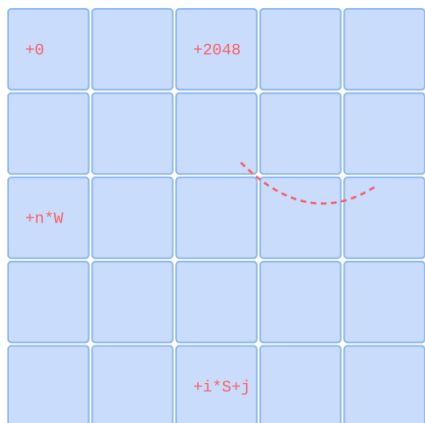
✓ 通过对性能无损的指令和写法进行充分封装, 实现极致性能和局部易用。

PyPTO-Pro: 易用性来源——Tile调度

- **Tile调度**: 核内所有操作都以二维tile视角, 简化Offset计算、指令参数填写

Manual

developer computes every offset & fills params

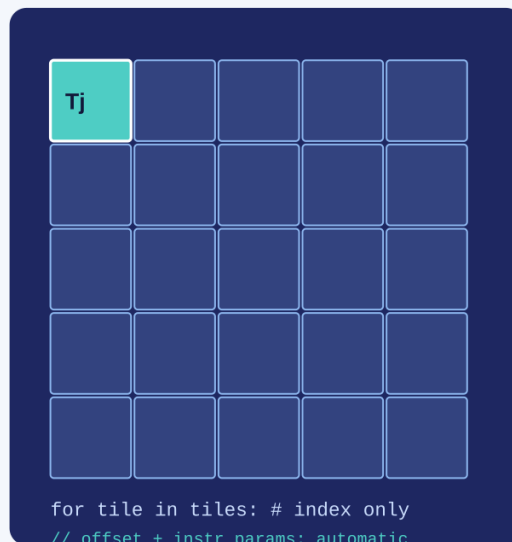


offset = base + i*stride + ...

pypto-pro

Tile-based scheduling

describe tiles — offsets & params auto-filled



变化点

- 用户基于Tile去描述计算和搬运而不是基于内存指针.
- 指令相关参数被简化成易于理解的Tile参数.
- 代码示例:

```
def compute(a_l1_tile, a_mat_4_buffer, b_mat_4_buffer,
            a_left_db, b_right_db, acc_db, a, b, out, i):
    for j in pl.range(0, b.shape[1] // 128, 1):
        a_l1_tile = a_mat_4_buffer.current()
        b_l1_tile = b_mat_4_buffer.next()
        pl.load_tile(b_l1_tile, b, [0, j])
        cur_a_left = a_left_db.next()
        pl.move(cur_a_left, a_l1_tile)
        cur_b_right = b_right_db.next()
        pl.move(cur_b_right, b_l1_tile)
        acc_tile = acc_db.next()
        pl.matmul(acc_tile, cur_a_left, cur_b_right)
        pl.store_tile(out, acc_tile, offset)
```

PyPTO-Pro: 易用性来源——核内同步简化

- **核内同步简化**: 利用tile_group机制简化核内同步和Buffer轮询

1 You write — request a tile block by role

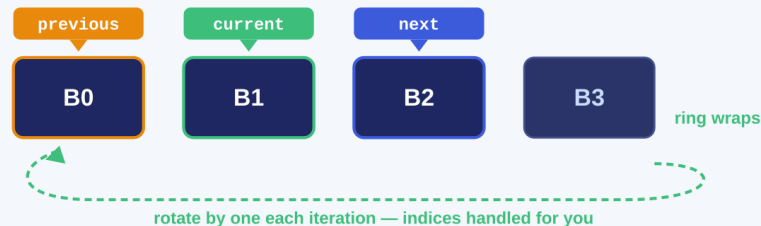
`previous()` `current()` `next()` → returns the tile

no buffer index, no $(i \% N)$ modulo arithmetic — just roles

framework maps roles → rotating buffers, and inserts sync

2 Framework handles rotation + synchronization

Multi-buffer ring — pointers rotate by one each iteration



✓ **Buffer rotation**
handled automatically — no manual buffer indices

✓ **Intra-core synchronization**
set_flag / wait_flag inserted for you

Manual synchronization



⚠ hand-placed event IDs — easy to deadlock or race

pypto-pro: auto-inserted



Dependencies inferred from data flow. The compiler places every barrier correctly — **no manual synchronization to write.**

变化点

- 用户不再需要手动指定多Buffer轮询的Index
- 用户不再需要插入核内同步，避免死锁和漏插同步
- 代码示例：

```
def compute(a_l1_tile, a_mat_4_buffer, b_mat_4_buffer,
            a_left_db, b_right_db, acc_db, a, b, out, i):
    for j in pl.range(0, b.shape[1] // 128, 1):
        a_l1_tile = a_mat_4_buffer.current()
        b_l1_tile = b_mat_4_buffer.next()
        pl.load_tile(b_l1_tile, b, [0, j])
        cur_a_left = a_left_db.next()
        pl.move(cur_a_left, a_l1_tile)
        cur_b_right = b_right_db.next()
        pl.move(cur_b_right, b_l1_tile)
        acc_tile = acc_db.next()
        pl.matmul(acc_tile, cur_a_left, cur_b_right)
        pl.store_tile(out, acc_tile, offset)
```

PyPTO-Pro: 易用性来源——核间流水编排

- 核间流水编排：开发者只需要关系计算逻辑，流水编排交给框架

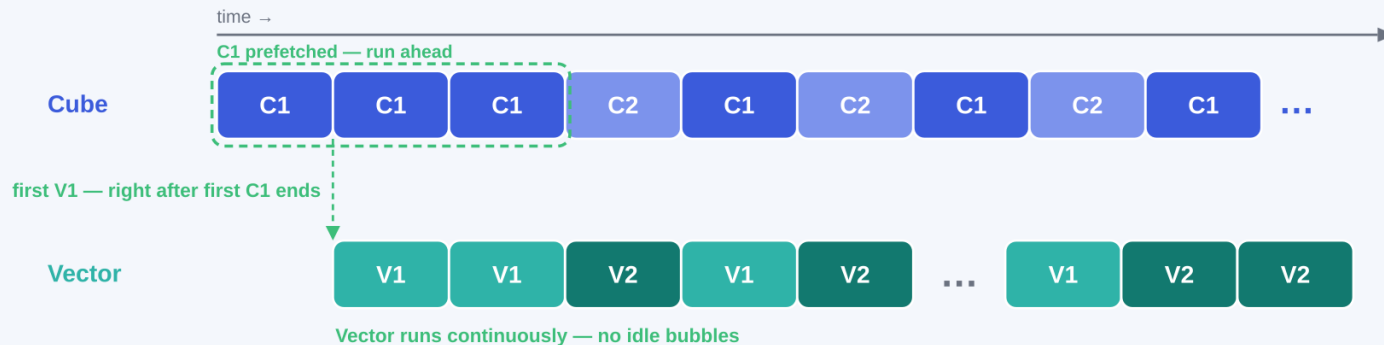
1 You write — plain sequential code



for i in range(n): C1; V1; C2; V2 # program order, no pipelining

framework auto-transforms
you rewrite nothing

2 Framework runs — software-pipelined



Emitted schedule:

Cube	C1	C1	C1	C2	C1	C2	C1	...	
Vector	V1	V1	V2	V1	V2	...	V1	V2	V2

变化点

- 用户只需要写串行的代码C1->V1->C2->V2
- 框架自动完成Preload流水编排以及核间同步插入
- 代码示例：

```
for qi in pl.range(core_id, sq_tiles, num_cores):
    sq_off = qi * TS
    with pl.section_cube():
        cur_q_slot = q_l1_db.next()
        for ki in pl.range(0, skv_tiles):
            with pl.section_cube():
                compute_qk(ki, sq_off, tick, q, k, cur_q_slot,
                           k_l1_db, left_db, right_db, acc_db, qk_vec_db)
            with pl.section_vector():
                compute_p(ki, tick, q_count, sub_id,
                         qk_vec_db, p_f16_db, p_f16_main_db, p_f16_back_db, p_mat_db,
                         global_max_rm_buf, global_sum_rm_buf, exp_corr_rm_fifo)
            with pl.section_cube():
                compute_pv(ki, tick, v, v_l1_db, p_mat_db,
                          left_db, right_db, acc_db, pv_vec_db)
            with pl.section_vector():
                compute_gu(ki, skv_tiles, tick, q_count, sq_off, row_off, o,
                          pv_vec_db, running_o_buf, o_f16_buf,
                          global_sum_rm_buf, exp_corr_rm_fifo)
            tick = tick + 1
        q_count = q_count + 1
```

CANN

PyPTO-Pro: 把“能写”变成“好写”

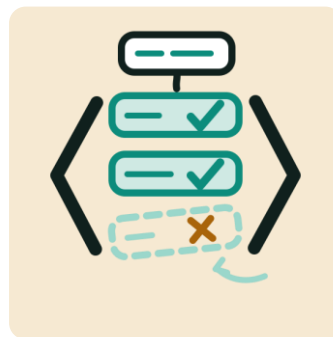


01

一份代码，JIT 调完直接发二进制

同一份 kernel 源码，不用为了发布再改写一遍。JIT 下快速迭代，调完配合 aclnn + tiling 直接二进制发布。

@pljit → aclnn + tiling



02

有效 tiling_key 用 Python 描述

哪些 tiling_key 组合有效，直接用 Python 写出来，无效的组合不必生成。不用再手工维护一张分支表。

tiling_key

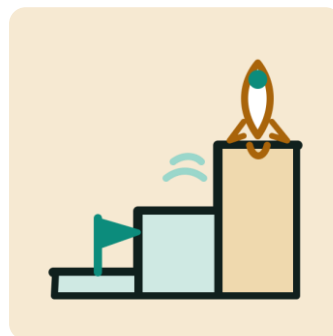


03

Python前端print / dump_tensor 全支持

想看哪一步就打哪一步，tensor 直接 dump 出来比对。

print / dump_tensor



04

先跑通功能，再手写 VF 压性能

完整的 tile 级算子库让功能先跑起来；性能不够时，热点上换成手写 vf.*。抽象是台阶，不是天花板。

tile 算子库 → vf.*

PyPTO：典型算子性能 & 代码量对比

- **极致性能**：pto-isa指令级提供了基于Tile的高性能编程范式，PyPTO充分利用这套指令级的优势，做到极致性能。
- **极简编码**：通过后端IR pass进行Buffer编排、核间核内流水编排、offset自动计算，代码量有大幅精简。

基于高性能pto-isa的指令级，0.9+理论性能

主流算子性能对比

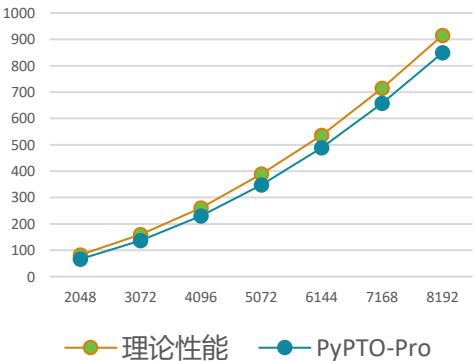
算子	理论性能(us)	PyPTO-Pro(us)	PyPTO-Tensor(us)
Matmul(MNK=4k)	324	330	668
Matmul(MN=8K, K=128)	47	49	217
LI(2K)	56	66	NA
LI(3K)	120	137	NA
LI(4K)	205	230	NA
LI(5K)	321	349	NA
LI(6K)	536	489	NA
LI(7K)	614	658	NA
LI(8K)	815	849	8000+
FA(2K)	15	15	NA
FA(4K)	24	26	47
FA(6K)	62	68	NA
FA(8K)	87	91	151
FA(16K)	326	344	NA
FA(32K)	1342	1394	NA
FA(64K)	5347	5557	NA
FA(128K)	21303	22336	NA

基于算子痛点，利用后端IR充分优化，精简代码量

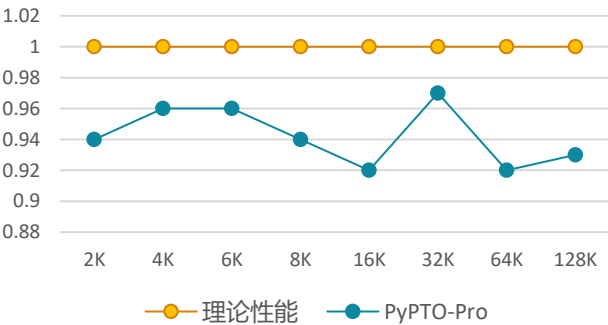
主流算子Kernel代码量（特定shape）对比

算子	PyPTO-Pro（行）	PyPTO-Tensor（行）
Matmul	50	5
LightningIndexer	500	180
FlashAttention	450	150

LI算子性能对比



FA算子性能对比

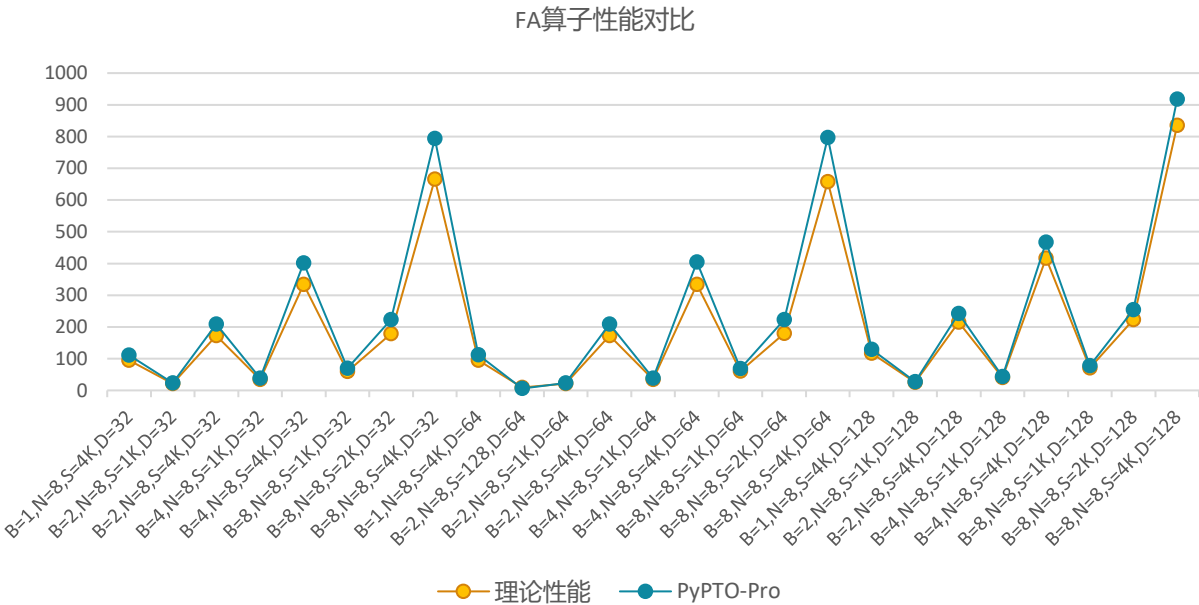


PyPTO: FlexAttention性能情况

	理论性能 (FA) (us)	PyPTO- Pro(FlexAttention)(us)	性能比
B=1,N=8,S=4K,D=32	94.47	111.2	0.84955
B=2,N=8,S=1K,D=32	21.436	23.4	0.916068
B=2,N=8,S=4K,D=32	172.37	209	0.824737
B=4,N=8,S=1K,D=32	34.591	38.3	0.903159
B=4,N=8,S=4K,D=32	333.99	401.2	0.832478
B=8,N=8,S=1K,D=32	59.43	69.4	0.85634
B=8,N=8,S=2K,D=32	178.44	223	0.800179
B=8,N=8,S=4K,D=32	666.2	793.6	0.839466
B=1,N=8,S=4K,D=64	94.47	111.8	0.844991
B=2,N=8,S=1K,D=64	21.742	23.7	0.917384
B=2,N=8,S=4K,D=64	172.69	208.7	0.827456
B=4,N=8,S=1K,D=64	34.829	38.5	0.904649
B=4,N=8,S=4K,D=64	333.62	404.2	0.825383
B=8,N=8,S=1K,D=64	60.47	69.1	0.875109
B=8,N=8,S=2K,D=64	179.26	222.8	0.804578
B=8,N=8,S=4K,D=64	658.2	796.5	0.826365
B=1,N=8,S=4K,D=128	117.62	129	0.911783
B=2,N=8,S=1K,D=128	25.291	27	0.936704
B=2,N=8,S=4K,D=128	215.14	242	0.889008
B=4,N=8,S=1K,D=128	40.331	43.6	0.925023
B=4,N=8,S=4K,D=128	416.59	467.2	0.891674
B=8,N=8,S=1K,D=128	71.14	78.2	0.909719
B=8,N=8,S=2K,D=128	222.83	254.2	0.876593
B=8,N=8,S=4K,D=128	835.57	918.2	0.910009

基于算子痛点，利用后端IR充分优化，精简代码量

模块	代码量
Cube侧搬运计算	60
Buffer分配	40
同步插入	0
指令映射	0
VF	600
Vector侧搬运计算	100
其他	200



先导版本已经在pypto仓开源发布

安装8月最新社区包即可体验：

- 代码仓路径：<https://gitcode.com/cann/pypto>
- 快速上手：https://gitcode.com/cann/pypto/tree/master/docs/zh/pypto_pro/tutorials/quick_start
- API文档：https://gitcode.com/cann/pypto/tree/master/docs/zh/pypto_pro/api
- Jit算子样例路径：https://gitcode.com/cann/pypto-gym/tree/master/src/pypto_gym/ops/pypto_pro
- 二进制发布的算子样例路径：
 - https://gitcode.com/cann/ops-transformer/tree/master/attention/dense_lightning_indexer_softmax_lse_v2
 - https://gitcode.com/cann/ops-transformer/tree/master/attention/sparse_flash_mla_softmax_l1_norm

目录

Part 1 Ascend C

Part 2 PyPTO

Part 3 CANNBot

CANNBot实践效果：端到端自动化，效率提升明显

性能实测 | 10 算子 950PR/A2 对比

算子	用例数	avg (950PR/A2)	耗时(min)
Shrink	350	2.66	91
CeluV2	146	1.62	92
Selu	96	1.53	89
Cosh	500	3.94	90
HardShrinkGrad	573	1.36	96
Real	473	1.28	85
Acosh	23	2.67	90
SoftPlus	30	2.76	120
HardSigmoidGrad	35	1.88	95
ReLU6Grad	589	1.33	89

注：性能比：>1 → 950PR 优，<1 → A2 优。589 为单算子最大用例数。

✓ 简单 / 中等算子 (L1/L2/L3)
Vector · Matmul 自动生成

● 复杂融合算子(L4)
CANNBot-DSL 攻关中，10+FA算子已验证，性能达到手写水平，大约3-5x效率提升。

○ 超复杂 MegaKernel(L5)
CANNBot-DSL 寻优是路径

300+

300+算子通过CANNBot已上库

2.1x

avg 性能比 950PR/A2 (10 算子均值)

8H

L1/L2算子的端到端开发，100% 一次通过编译与真机精度的平均耗时，性能0.3x理论。（不含调优）

CANNBot算子生成提示词

CANNBot

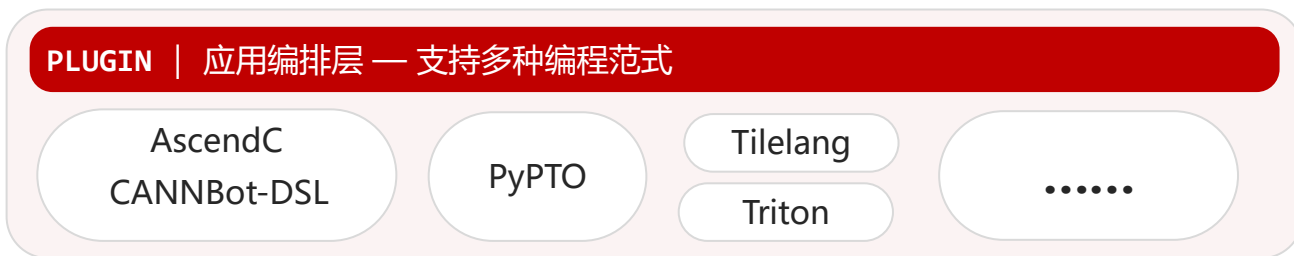
```
/ops-direct-invoke-flash算子名:swiglu;公式: $y=x1* \text{silu}(x2)$ ,其中  $\text{silu}(x)=x*\text{sigmoid}(x)$ ;输入: x1,x2共2个,道元素运算,shape相同dtype相同;输出:y,与输入shape相同,dtype相同;数据类型:float32
```

Build · GLM-5.3 CANNBot

tab agents ctrl+p commands

CANNBot三层解耦：Plugin 编排、Agent 执行、Skill 承载知识

沿「补知识 × 建信任」双线组织，覆盖算子开发全生命周期——Skill 可复用、Agent 可替换、Plugin 可编排。



编 排



绑 定



双线 | 补知识 × 建信任

- **补知识:** cannbotskills 封装领域知识
cannbot-knowledge 承载检索态资产
- **建信任:** cann-bench 硬件基准验收
cannbot-sentry 过程观测审计

CANNBench 昇腾算子评测基准

从统一任务规格出发，通过精度校验，在真实 Ascend NPU 上采集性能，并在排行榜中比较结果。



<https://cannbench.com/>

[https:// gitcode.com/cann/cannbot-skills](https://gitcode.com/cann/cannbot-skills)

CANN

CANNBot Harness: 不限制模型能力，而是把能力精确导向目标

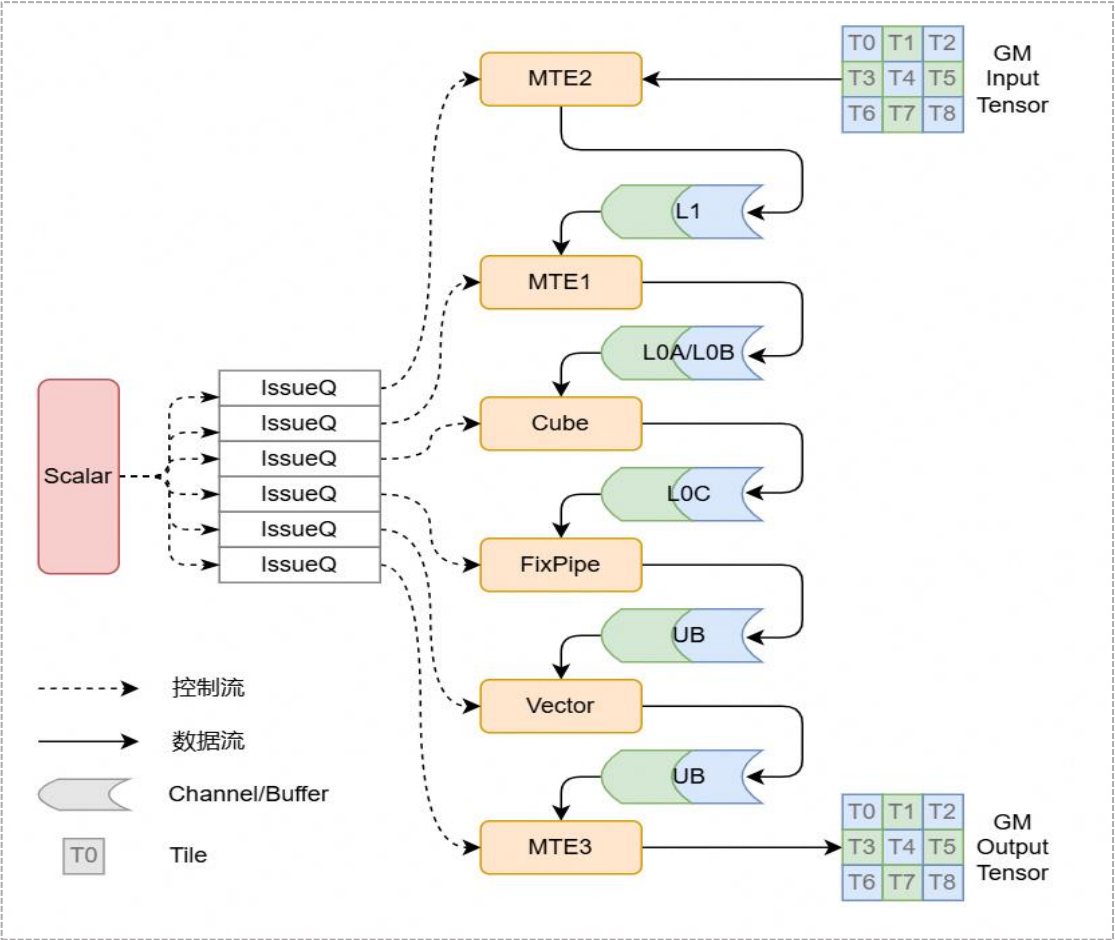
Harness = harness-core (推进) + harness-cp (把关) ——LLM 的不确定性锁在单步内，跨步推进 / 重试 / 恢复皆是编排器的确定性决策。



CANNBot-DSL：昇腾亲和Agent Friendly编程范式探索

建模
抽象

- 1. 硬件架构：昇腾芯片是基于Tile的异步数据流的SIMD架构，具有统一的MainScalar单元，异步执行的计算和搬运单元
- 2. 协程模型：Scalar作为协程调度单元，计算和搬运流水线作为异步协程执行单元，核内buffer作为异步协程数据流管道
- 3. 领域抽象：基于Channel、Buffer、Tensor、Layout领域抽象来表达算子的流水编程、Buffer管理、Tiling切分以及Tile调度



流水编排 Buffer管理 Tiling切分 Tile调度

领域
抽象

Channel Buffer Tensor Layout

- 基于协程模型和领域抽象构建出昇腾亲和的编程范式：
- 算子Kernel编程操作的主体从裸指针转变为Tensor；
 - 基于Layout表达数据Tile块的切分以及Tile块在核间/核内的调度；
 - 使用Channel来串联算子异步的计算和搬运操作；
 - Channel作为异步操作数据流的管道，内部自动处理异步执行单元间的同步插入以及Buffer的自动切换

探索将芯片异步并行计算抽象为面向软件串行计算

探索将串行复杂的计算过程抽象为软件组件简化计算表达

CANNBot-DSL: 性能无损前提下的易用性提升

原始公式并不长

```
— Inputs
Q ∈ ℝ^(N × d)      # query
K ∈ ℝ^(M × d)      # key
V ∈ ℝ^(M × d)      # value
M_mask ∈ {0, -∞}^(N × M) # 可选 (causal / window)
scale = 1 / √d

— Outputs
O ∈ ℝ^(N × d)      # attention output
L ∈ ℝ^N            # log-sum-exp (反向用)

— Tiling
N = T_r · B_r, Q split → Q_1, ..., Q_{T_r}
M = T_c · B_c, K split → K_1, ..., K_{T_c}
V split → V_1, ..., V_{T_c}

— 主算法 (FlashAttention-2)
for i = 1 ... T_r:
  Load Q_i ∈ ℝ^(B_r × d)
  Init m_i^a(0) = -∞
  ℓ_i^a(0) = 0
  O_i^a(0) = 0 ∈ ℝ^(B_r × d)
  for j = 1 ... T_c:
    Load K_j, V_j ∈ ℝ^(B_c × d)
    (1) S_i^a(j) = scale · Q_i · K_j^T + M_mask
    (2) m_i^a(j) = max(m_i^a(j-1), rowmax(S_i^a(j)))
    (3) P_i^a(j) = exp(S_i^a(j) - m_i^a(j))
    (4) α_i^a(j) = exp(m_i^a(j-1) - m_i^a(j)) # rescale
    (5) ℓ_i^a(j) = α_i^a(j) · ℓ_i^a(j-1) + rowsum(P_i^a(j))
    (6) O_i^a(j) = α_i^a(j) · O_i^a(j-1) + P_i^a(j) · V_j
    (7) O_i = O_i^a(T_c) / ℓ_i^a(T_c) # 最终归一化
    (8) L_i = m_i^a(T_c) + log(ℓ_i^a(T_c))
  Return O = stack(O_1, ..., O_{T_r}), L = stack(L_1, ..., L_{T_r})
```

可读公式 ~38 行

为完成计算引入大量的细节

```
// fa_arch35.cpp ---- FlashAttentionScore kernel for Ascend dav-3510
template<typename T, typename U, FaShape SHAPE_T,
        CausalMode C_MODE, int B_r, int B_c, int D,
        int CUBE_TILE_M = 128, int CUBE_TILE_N = 128>
class FlashAttentionScoreKernel {
public:
  __aicore__ void Init(__gm__ const T* q, __gm__ const T* k,
                      __gm__ const T* v, __gm__ const T* atten_mask,
                      __gm__ U* softmax_max, __gm__ U* softmax_sum,
                      __gm__ T* out, const FaTilingData* tiling) {
    /* ... 100+ 行代码拷贝 / 边界判定 / mode 分支 ... */
  }
  __aicore__ void ProcessOneQTile(int b, int h, int m) {
    // — 1) Q 字节偏移: 手算 (BNSD)
    const int64_t q_gm_off =
      (int64_t)b * tiling_n1 * tiling_s1 * tiling_d * sizeof(T)
      + (int64_t)h * tiling_s1 * tiling_d * sizeof(T)
      + (int64_t)m * B_r * tiling_d * sizeof(T);
    // — 2) NZ on L1: 手算 (n1, m1, m0, n0) + n1_pad bank pad
    constexpr int M0 = 16;
    constexpr int N0 = 32 / sizeof(T);
    constexpr int n1 = (D + N0 - 1) / N0;
    constexpr int m1 = (B_r + M0 - 1) / M0;
    constexpr int n1_pad = 32 / sizeof(T);
    constexpr int q_l1_slot_elems = (m1 * M0 * N0 + n1_pad) * n1;
    const int q_l1_off = cur_q_l1_slot_
      * q_l1_slot_elems * (int)sizeof(T);
    // — 3) GM -> L1 nd2nz (8 参 DMA, AIC 段)
    if (g_coreType == AIC) {
      asc_get_buf(PIPE_MTE2, AIC_L1_Q_BUF + cur_q_l1_slot_);
      asc_copy_gm2l1_nd2nz(
        /*l1_buf*/ q_l1_,
        /*l1_offset*/ q_l1_off,
        /*gm_buf*/ q_gm_ + q_gm_off,
        /*gm_offset*/ 0,
        /*n_value*/ B_r,
        /*d_value*/ D,
        /*src_row_stride*/ tiling_s1 * tiling_d * sizeof(T),
        /*nd2nz*/ true,
        /*pad*/ 0);
      asc_sync_intra_arrive(PIPE_MTE2,
        AIC_Q_L1_READY + cur_q_l1_slot_);
      asc_release_buf(PIPE_MTE2, AIC_L1_Q_BUF + cur_q_l1_slot_);
      cur_q_l1_slot_ ^= 1; // ping-pong 游标
    }
    // — 4) K->L1 / V->L1 / L1->L0A / L1->L0B / mmad / L0C->UB
    // online-softmax (4 个 row-loop) / P->L1 / mmad PV /
    // L0C->GM / finalize / store max,sum / sync 配对 ....
    // 每个方向 30~60 行, AIC/AIV 各一份, 全套 ≈ 2,000 行.
  }
private:
  // ← 这一坨游标全部要手维护
  int cur_q_l1_slot_ = 0, cur_k_l1_slot_ = 0, cur_v_l1_slot_ = 0;
  int cur_qk_ub_slot_ = 0, cur_pv_ub_slot_ = 0, cur_p_l1_slot_ = 0;
  int cur_l0c_slot_ = 0, cur_softmax_slot_ = 0, cur_sm_exp_slot_ = 0;
};
```

Ascend C ~2,000 行

arch35 真实工程 (init / process / online-softmax / split-mix)

前端表达&IR对细节进行封装

① Pythonic表达: AST Rewrite + Tracing

- 保留程序的控制流 (*for/if/...*)
- 支持全量*Python*的语法表达

② 核心抽象: Tensor + Layout + Channel

- 屏蔽用户指令参数的显示拼装
- 取代*Nbuffer* 同步ID的手动游标操作

③ Layout代数: ceil_div + idx2crd + ...

- 实现*Tiling*切分方案的正向表达
- 屏蔽用户显示的索引和坐标的转换

④ Tensor切分: tile_view

- 屏蔽用户复杂*layout*下物理地址*offset*的显示计算
- 屏蔽用户非对齐/尾块的处理

⑤ Channel: 流水编排

- 使用*Channel*显式表达数据流
- 自动插入同步

对影响易用性的点进行抽象/建模/封装

提升编写代码的易用性

```
■ for l = 1 ... T_r:
  ■ Load Q_l; init m, ℓ, O = -∞, 0, 0
  ■ for j = 1 ... T_c:
    ■ Load K_j, V_j
    ■ (1) S = scale · Q_i · K_j^T + M_mask
    ■ online softmax (步骤 2~5)
    ■ (2) m = max(m_old, rowmax(S))
    ■
    ■ (3) P = exp(S - m)
    ■
    ■ (4) α = exp(m_old - m)
    ■ (5) ℓ = α · ℓ_old + rowsum(P)
    ■ cast P to fp16, P → L1 (NZ)
    ■ (6) O = α · O_old + P · V_j
    ■
    ■ (7) O_i = O_i / ℓ_i
```

```
■ for tile_idx in range(block_idx, total, block_num):
  ■ matmul.load_q(q_l1, q_tile_gm)
  ■ for n_idx in range(kv_tile_num):
    ■ matmul.load_k(k_l1, k_tile_gm)
    ■ matmul.mm_qk(l0c, q_l1, k_l1)
    ■ with vf(outputs=[max_sum, exp_p]):
      ■ muls(qk, qk, scale)
      ■ reduce_max(m_new, qk, axis=1);
  ■ vmax(m_new, m_old, m_new)
  ■ sub(qk, qk, expand(m_new))
  ■ exp(qk, qk)
  ■ sub(a, m_old, m_new); exp(a, a)
  ■ mul(ℓ, ℓ, a); reduce_sum(s_new, qk);
  ■ add(ℓ, ℓ, s_new)
  ■ cast(p_fp16, qk); mem_copy(p_l1, p_fp16,
  engine=nd2nz)
  ■ matmul.mm_pv(l0c, p_l1, v_l1)
  ■ mul(0, 0, expand(a)); add(0, 0, pv_ub)
  ■ div(0, 0, expand(ℓ)); cast(0_fp16, 0);
  mem_copy(out_gm, 0_fp16)
```

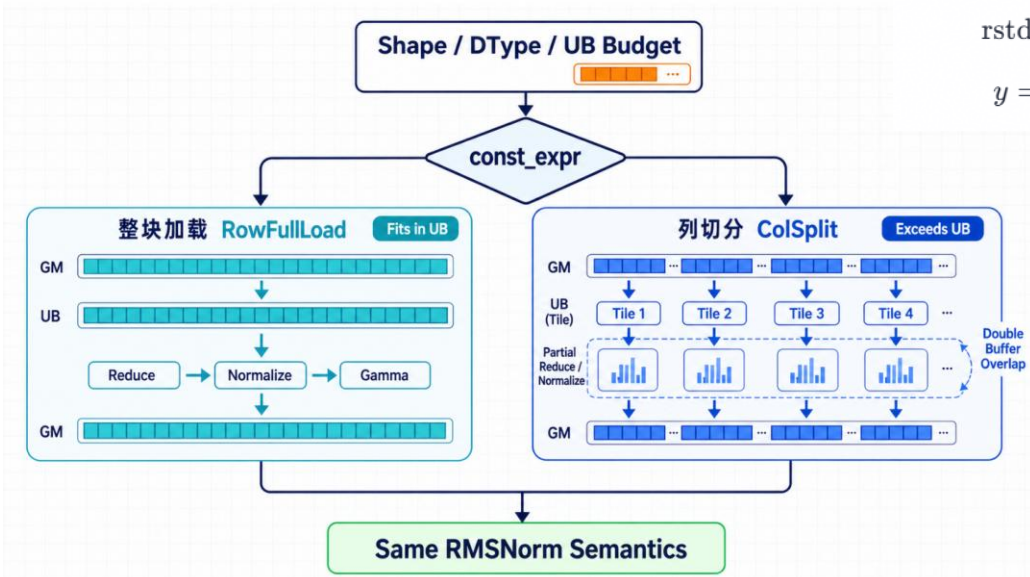
CANNBot-DSL ~700 行

案例：一天完成RMSNorm算子生成与调优

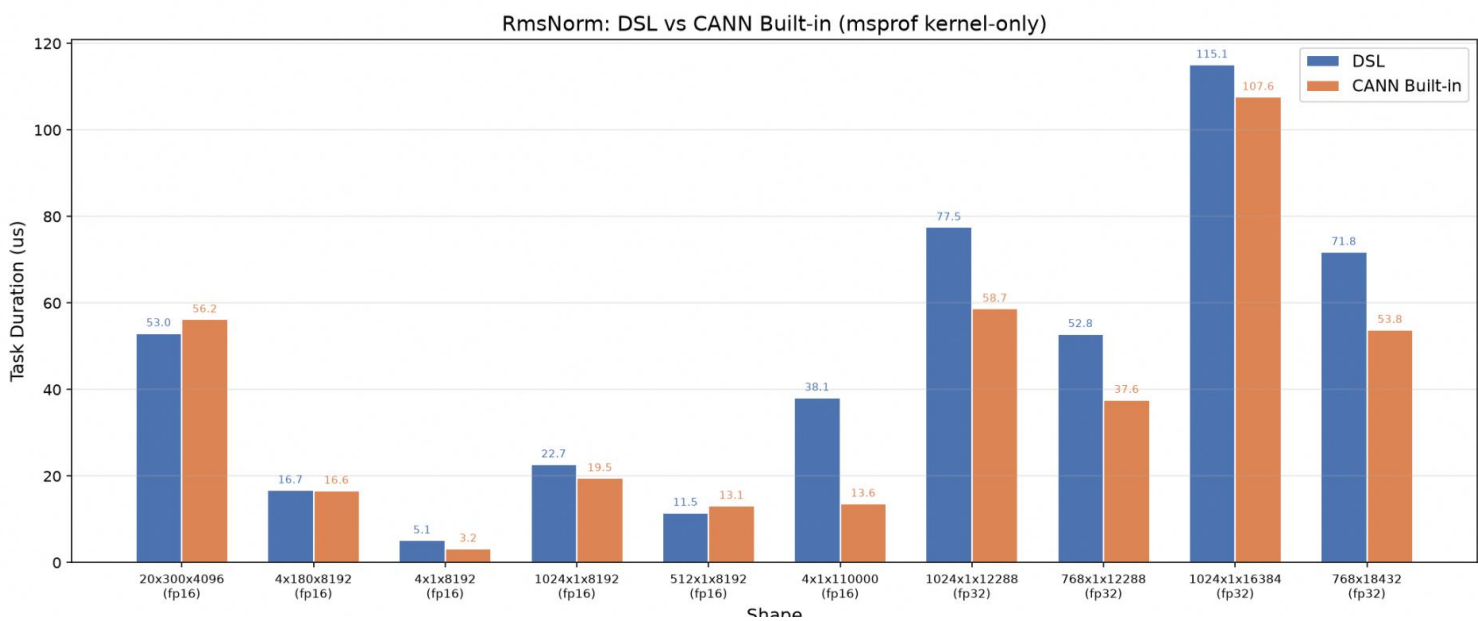
- 编译期模板选择
- 流水深度和 Unroll 独立参数调优

```
@jit
def run(self, gm_x, gm_gamma, gm_y, gm_rstd, eps):
    num_col = gm_x.shape[1]
    block_dim = math.ceil(num_row / block_factor)
    if const_expr(self._is_row_full_load(num_col,
self.dtype)):
        op = RowFullLoadKernel(num_col, self.dtype)
    else:
        op = ColSplitKernel(num_col, self.dtype)
    op[block_dim](gm_x, gm_gamma, gm_y, gm_rstd, eps)
```

```
self._x_db_ch = Channel(
    MemLoc.UB, shape=(1, self._w), dtype=buf_dtype,
    depth=2
)
for seg in cannbotdsl.range(self._num_seg, unroll=2):
    mask, remaining = update_mask(remaining,
    elem_bits=32)
```

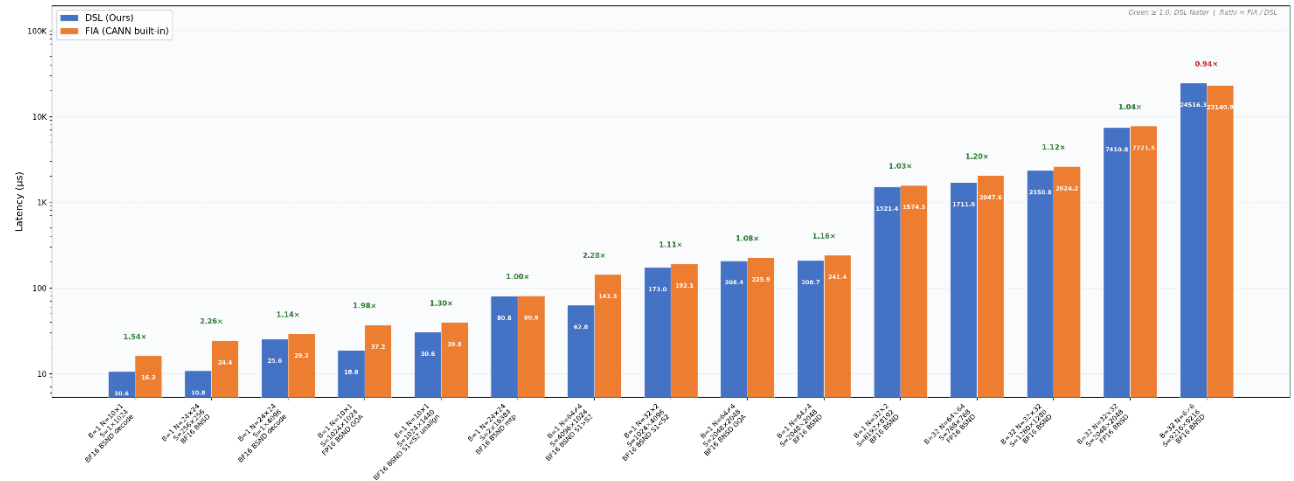


$$\text{rstd} = \frac{1}{\sqrt{x^2 + \epsilon}}$$
$$y = x \cdot \text{rstd} \cdot \gamma$$



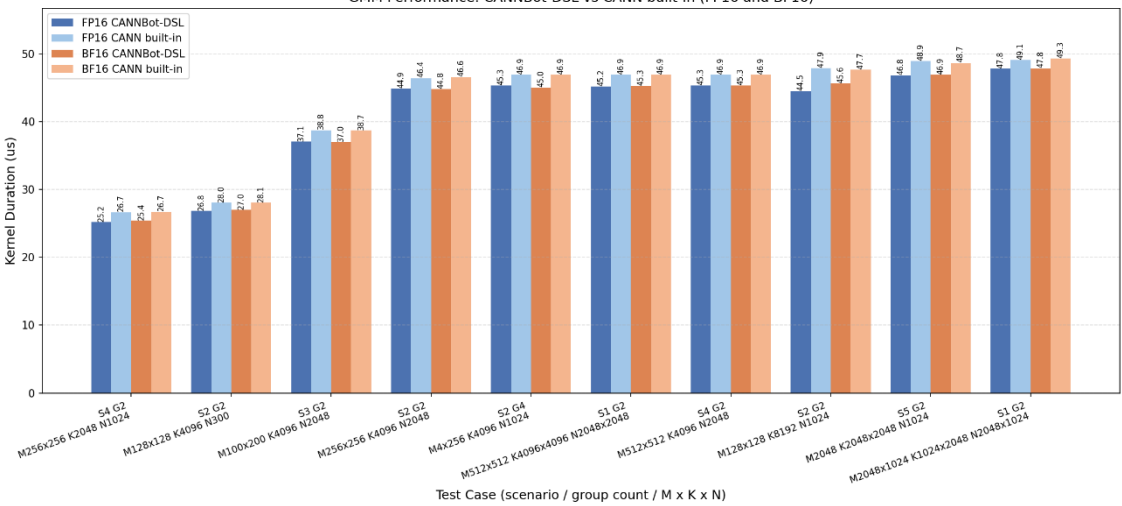
基于CANNBot-DSL的典型算子性能

Flash Attention Performance Comparison: DSL vs FIA



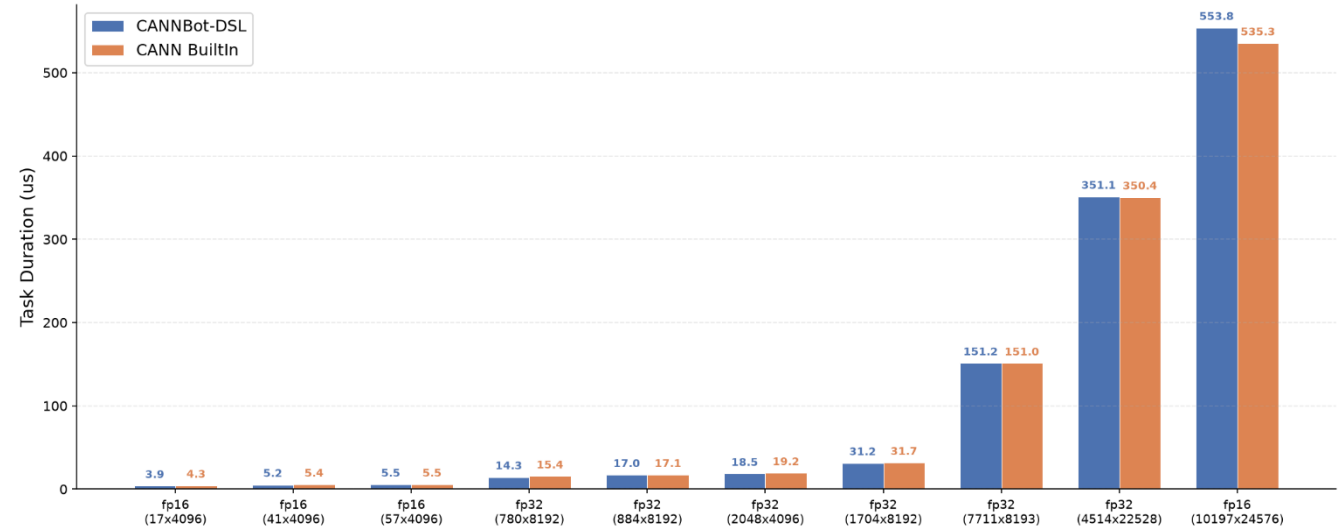
Flash Attention

GMM Performance: CANNBot-DSL vs CANN built-in (FP16 and BF16)

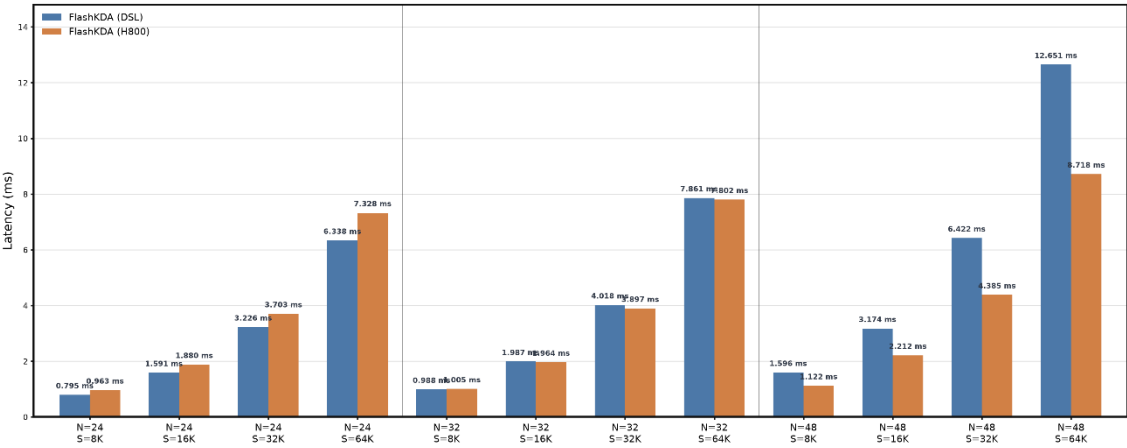


Grouped Matmul

RmsNorm Performance: CANNBot-DSL vs CANN BuiltIn



RMSNorm



FlashKDA

Agent-Friendly 范式探索——基于CANNBot的高性能复杂算子生成

昇腾CANN 2026年8月5日 20:13 江苏



Agent-Friendly范式探索: 基于CANNBot的高性能复杂算子生成

昇腾CANN · 08-07

直播:B站视频

用CANNBot去写高性能复杂算子时,拦住它的从来不是算法,而是一堆"天生对Agent不友好"的细节:为榨干多级流水而生的NBuffer、核内核间稍有不慎就出错的同步原语、各种Layout堆出来的复杂地址计算、还有因为极致性能而引入的私有格式。它们与算法无关,但也是Agent生成复杂算子最容易出错的主要原因。面对这些复杂的细节,当前的大模型能力、提示词和skills还无法完全解决, Harness工程能够保证代码的部分正确性, 但面对复杂算子, 依然无法达成高性能的要求。这里并不是CANNBot不够聪明, 而是不该把这些细节交到它手里。

换个角度看,这也给破局指明了方向: 真正的Agent-Friendly,不是靠更强的提示词把 Agent "教会", 而是为它重新划定边界,只把它擅长的留下, 把不擅长的交给软件框架。CANNBot-DSL正是基于此考虑:把那些复杂、易错、却又不左右性能的动作逐一建模,沉淀为一层架设在昇腾稳定 Ascend C之上的软件抽象, 让CANNBot彻底从繁冗细节中解放出来, 回到算法与数据流的主场, 心无旁骛,尽展所长。

Agent生成复杂高性能算子困境的破局之道

为突破Agent开发复杂高性能算子的各类难题, CANNBot-DSL方案通过昇腾硬件友好的协程范式、框架自动管理同步、Agent友好的API封装、尾块自动处理、性能优化关键因子拆解和求解器寻优等多种机制攻克开发痛点, 从稳定性、开发难度、硬件适配、运算效率多方面带来增益。

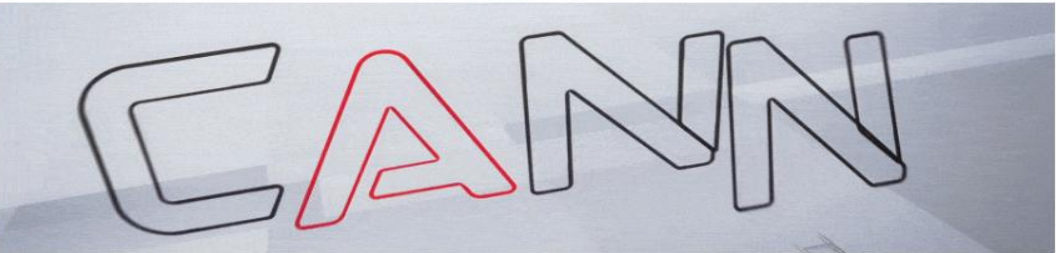
以上这些困境, CANNBot-DSL破局之道背后都是同一套逻辑:凡是与算法无关, 又不左右性能的繁

CANN 昇腾CANN 赞 分享 推荐 写留言

https://mp.weixin.qq.com/s/c2gPyhFViBrK6GZjoJ87oQ

Kimi K3 CANN推理实践：从Day0快速适配到性能持续优化

昇腾CANN 2026年8月15日 20:33 江苏



”

Moonshot AI开源Kimi K3模型, CANN已完成Day0适配, 支持昇腾950系列部署; 在此基础上CANN持续优化整网部署, 依托CANNBot-DSL完成KDA与Attention Residual算子融合, 叠加MegaMoE大EP优化以及DSpark投机推理, 进一步提升部署性能, 相关代码已开源至cann-recipes-infer仓库。

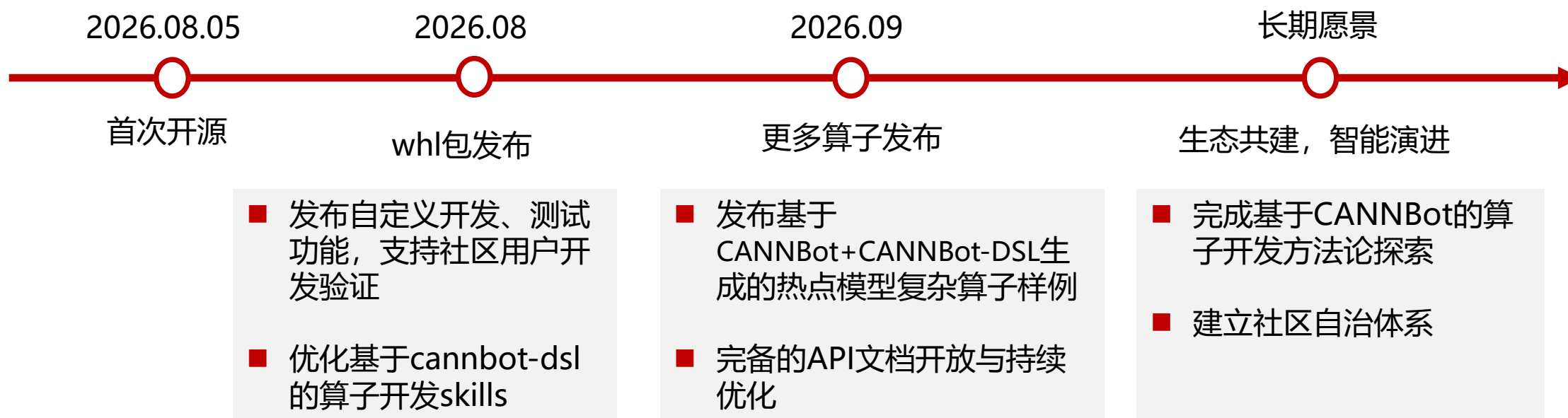
背景: Kimi K3 高性能推理的关键挑战

通过对 Kimi K3 整网执行链路分析, 关键计算仍有进一步融合优化空间: KDA 前处理、核心计算与状态递推之间存在较多中间张量读写; Attention Residual 需要多次访问历史 Block; 896 专家 MoE 的 Token 通信与专家计算开销较大; 针对这些问题, 本次优化扩大 KDA 和 Attention Residual 的融合范围, 引入 MegaMoE 优化, 并通过 DSpark 进一步提升解码效率。

CANN 昇腾CANN 赞 分享 推荐 写留言

https://mp.weixin.qq.com/s/HZF3MLqHiF_5R5Dgt4enOQ

CANNBot-DSL Roadmap



代码仓地址: <https://gitcode.com/cann/cannbot-dsl>

欢迎前往仓库查阅体验!



CANN

总结:

Ascend C

极致性能的编程范式

PyPTO

Python编程范式
具备易用性和性能

CANNBot

Agent时代的编程入口

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN